This lab builds off of lab 3, with the following goals:

1. Finish implementation of the two scheduling algorithms from lab 3
2. Solidify your observations and pros/cons of the scheduling algorithms
3. Implement the MLFQ algorithm (multilevel feedback queue)

1. Finish implementation of the two scheduling algorithms from lab 3

I implemented both scheduling algorithms in lab 3.

2. Solidify your observations and pros/cons of the scheduling algorithms

I finished this in the previous lab as well.

3. Implement the MLFQ algorithm (multilevel feedback queue)

Before we jump into the implementation, we're instructed to do some reading of the chapters here, and answer some questions:

- What is the runtime of a lookup in an array?
  - O(n)
- What is the runtime of a lookup in an arraylist?
  - O(n)
- How is an array stored in memory?
  - Array is stored as consecutive memory of static size
- How is an arraylist stored in memory?
  - Arraylist is stored as consecutive memory of dynamic size
- Why was the hashmap data structure introduced?
  - To pass the O(n) issues with arrays/arraylists by using hash function to achieve O(1) lookup.

MLFQ:

- Why do you think the quantum times differ?
  - To optimize fairness and system performance
- What do you think the boost time is for?
  - To allow processes which have been stuck in low level queues to have a chance at high priority again

- Why is the lowest priority queue using FCFS?
    - Because the processes in this queue are known to take a long time, and a majority of processes should be completed before hitting the FCFS queue, it's okay if a low priority process gets stuck waiting for a while.

We're given the following rules for the MLFQ algorithm:

Let's implement the MLFQ algorithm now using these rules.

MLFQ Implementation

First, let's add MLFQ as an option on our dropdown menu:

```
option = new JComboBox(new String[]{"FCFS", "SJF", "SRT", "PSN", "PSP",
"RR", "LJF", "DPSN", "SPSN", "MLFQ"});
```

I also need to add a case to the switch statement:

```
case "MLFQ":
     String tq2 = JOptionPane.showInputDialog("Time Quantum");
     if (tq2 == null) {
          return;
     }
     scheduler = new MultiLevelFeedbackQueue();
     scheduler.setTimeQuantum(Integer.parseInt(tq2));
     break;
```

This will prompt the user to insert a quantum time value upon selecting MLFQ.

Now, I'll make a new file `MultiLevelFeedbackQueue.java` and start implementing.

```java
import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

//

public class MultiLevelFeedbackQueue extends CPUScheduler{
    @Override
    public void process()
    {
        Collections.sort(this.getRows(), (Object o1, Object o2) -> {
            if (((Row) o1).getArrivalTime() == ((Row) o2).getArrivalTime())
            {
                return 0;
            }
            else if (((Row) o1).getArrivalTime() < ((Row) o2).getArrivalTime())
            {
                return -1;
            }
            else
            {
                return 1;
            }
        });


        List<Row> q1 = new ArrayList();
        List<Row> q2 = new ArrayList();
        List<Row> q3 = new ArrayList();

        List<Row> rows = Utility.deepCopy(this.getRows());
        int time = rows.get(index: 0).getArrivalTime();
        int timeQuantum1 = this.getTimeQuantum();
        int timeQuantum2 = timeQuantum1 * 2;
        int lastBoost = time;
        int boostTime = timeQuantum1 * 10;


        for (int i = 0; i < rows.size(); i++){
            if (rows.get(i).getArrivalTime() <= time)
            {
                q1.add(rows.remove(i));
                i--;
            }
        }
```

```java
while (!q1.isEmpty() || !q2.isEmpty() || !q3.isEmpty() || !rows.isEmpty())
{
    if (q1.isEmpty() && q2.isEmpty() && q3.isEmpty())
    {
        time = rows.get(index: 0).getArrivalTime();
        lastBoost = time;
    }

    for (int i = 0; i < rows.size(); i++)
    {
        if (rows.get(i).getArrivalTime() <= time)
        {
            q1.add(rows.remove(i));
            i--;
        }
    }

     if (time - lastBoost >= boostTime)
    {
        q1.addAll(q2);
        q1.addAll(q3);
        q2.clear();
        q3.clear();
        lastBoost = time;
    }

    List<Row> currentQueue;
    int quantum;

    if (!q1.isEmpty())
    {
        currentQueue = q1;
        quantum = timeQuantum1;
    }
    else if (!q2.isEmpty())
    {
        currentQueue = q2;
        quantum = timeQuantum2;
    }
    else
    {
        currentQueue = q3;
        quantum = Integer.MAX_VALUE;
    }
```

```java
            Row row = currentQueue.remove(index: 0);
            int bt = (row.getBurstTime() <= quantum) ? row.getBurstTime() : quantum;
            this.getTimeline().add(new Event(row.getProcessName(), time, time + bt));
            time += bt;

            for (int i = 0; i < rows.size(); i++)
            {
                if (rows.get(i).getArrivalTime() <= time)
                {
                    q1.add(rows.remove(i));
                    i--;
                }
            }

            if (row.getBurstTime() > quantum)
            {
                row.setBurstTime(row.getBurstTime() - quantum);

                if (currentQueue == q1)
                {
                    q2.add(row);
                }
                else if (currentQueue == q2)
                {
                    q3.add(row);
                }
                else
                {
                    q3.add(row);
                }
            }
        }

            Map map = new HashMap();
        for (Row row : this.getRows())
        {
            map.clear();
            for (Event event : this.getTimeline())
            {
                if (event.getProcessName().equals(row.getProcessName()))
                {
                    if (map.containsKey(event.getProcessName()))
                    {
                        int w = event.getStartTime() - (int) map.get(event.getProcessName());
                        row.setWaitingTime(row.getWaitingTime() + w);
```

```
                    }
                    else
                    {
                        row.setWaitingTime(event.getStartTime() - row.getArrivalTime());
                    }
                    map.put(event.getProcessName(), event.getFinishTime());
                }
            }
            row.setTurnaroundTime(row.getWaitingTime() + row.getBurstTime());
        }
    }
}
```

I just screenshotted my code because copy pasting the text was messing up indentation.